

알고리즘분석 핵심 정리

2026년 3학년 1학기 | 두원공과대학교 컴퓨터공학과

담당: 김영오 교수

1. 알고리즘 개요

1-1. 알고리즘이란?

- 주어진 문제를 해결하기 위해 명확하게 정의된 유한한 단계의 명령절차
- 페르시아 수학자 알과리즈미의 이름에서 유래

1-2. 알고리즘의 5 가지 조건

- 0 개 이상의 외부 자료가 제공되어야 함
- 최소 1 개 이상의 결과를 도출해야 함
- 각 단계는 모호하지 않고 명확해야 함
- 유한한 단계 후 반드시 종료되어야 함
- 모든 명령은 실행 가능하고 단순해야 함

1-3. 시간복잡도 — Big-O 표기

다항식에서 최고 차수 항만 취한 뒤, 그 항의 계수를 제거하여 표현

예) $f(n) = 2n^2 + 3n + 5 \rightarrow O(n^2)$

표기	이름	설명
$O(1)$	상수 시간	입력 크기와 무관하게 일정
$O(\log n)$	로그 시간	입력이 클수록 증가폭 감소
$O(n)$	선형 시간	입력 크기에 비례
$O(n \log n)$	로그 선형	정렬 알고리즘 평균
$O(n^2)$	2차 시간	이중 반복문
$O(2^n)$	지수 시간	지수적 증가, 매우 느림

1-4. 분석 종류

- 수행시간 상한 $\rightarrow$ Big-O 사용
- 입력의 균등 분포(Uniform Distribution) 가정

- 최적(optimal) 알고리즘을 찾는 데 활용

2. 자료구조 분류

2-1. 선형 vs 비선형

구분	특징	예시
선형	1:1 관계, 단일 레벨, 탐색 쉬움	배열, 연결리스트, 스택, 큐
비선형	1:n 또는 n:n 관계, 멀티레벨, 메모리 효율적	트리, 그래프

2-2. 배열 (Array)

- 연속된 메모리 공간에 순차적으로 저장
- 배열의 크기는 고정
- $O(1)$ — 인덱스로 바로 접근
- $O(n)$ — 뒤 요소를 밀거나 당겨야 함

2-3. 연결 리스트 (Linked List)

- 메모리 공간에 불연속적으로 저장, 포인터로 연결
- 첫 번째 노드: head, 마지막 노드: tail
- $O(1)$
- $O(n)$
- 검색 성능이 배열보다 낮음

2-4. 스택 (Stack)

- LIFO (Last-In-First-Out) — 후입선출
- 요소를 맨 위에 추가
- 맨 위 요소를 제거하고 반환
- 짝 찬 스택에 push
- 빈 스택에 pop
- 웹 뒤로가기, undo, 괄호 검사, 후위 표기법 계산

2-5. 큐 (Queue)

- FIFO (First-In-First-Out) — 선입선출
- rear(뒤)에 삽입
- front(앞)에서 삭제
- 프로세스 관리, BFS, 캐시 구현

3. Python 자료형

3-1. 기본 자료형 분류

자료형	타입	변경 가능	예시
정수	int	불가 (Immutable)	34, -2, 0
실수	float	불가	3.14, -0.5
문자열	str	불가	'hello', '안녕'
진위	bool	불가	True, False
복소수	complex	불가	8+3j
바이트	bytes	불가	b'abcd'
리스트	list	가능 (Mutable)	[1, 2, 3]
딕셔너리	dict	가능	{'key': 'val'}
튜플	tuple	불가	(1, 2, 3)
집합	set	가능	{1, 2, 3}

3-2. 주요 개념

- 짧은 문자열은 동일 객체 공유 → **is vs ==** 구분 필요
- **str=Unicode** 문자열 / **bytes=Binary** 숫자열. **encode()/decode()**로 상호변환
- **or** → 왼쪽 **False** 면 오른쪽 반환 / **and** → 왼쪽 **True** 면 오른쪽 반환
- **id()**는 메모리 주소, **is** 는 동일 객체, **==**는 값 비교

3-3. 논리·비교 연산자

연산자	의미
$x > y / x < y / x \geq y / x \leq y$	크다/작다/이상/이하
$x == y$	같다
$x != y$	같지 않다
$x \text{ and } y$	논리곱 (둘 다 True)
$x \text{ or } y$	논리합 (하나라도 True)
$\text{not } x$	부정

3-4. 리스트 핵심 메서드

메서드	설명
<code>a.append(x)</code>	끝에 요소 추가

<code>a.insert(i, x)</code>	i번째 위치에 삽입
<code>a.extend(b)</code>	리스트 <code>b</code> 를 <code>a</code> 에 이어붙임
<code>a.pop(i)</code>	i번째 요소 제거 후 반환
<code>a.remove(x)</code>	첫 번째 <code>x</code> 값 제거
<code>a.sort()</code>	오름차순 정렬 (in-place)
<code>sorted(a, reverse=True)</code>	내림차순 정렬 (새 리스트 반환)
<code>a.index(x)</code>	<code>x</code> 의 인덱스 반환
<code>a.count(x)</code>	<code>x</code> 의 개수 반환

4. NumPy / Pandas

4-1. NumPy — ndarray

- **Scalar** → **Vector** → **Matrix** → **Tensor**
- 배열의 형태 (예: 1D=(4,) / 2D=(2,3) / 3D=(4,3,2))
- 전체 원소의 수
- 저장된 데이터 타입
- 0=행 방향, 1=열 방향

주요 활용 예시

```
import numpy as np
a = np.array(range(1, 11))      # [1, 2, ..., 10]
a + a[::-1]                    # [11, 11, ..., 11]
```

4-2. Pandas — Series & DataFrame

- 인덱스(index) + 값(value)으로 구성된 1차원 자료구조
- 행과 열로 이루어진 2차원 자료구조 (여러 **Series**의 결합)

5. 기초 알고리즘 실습 코드

5-1. 1부터 N까지 합 구하기

for 루프 방식

```
result = 0
for i in range(1, n+1):
    result += i
```

내장함수 방식

```
result = sum(range(1, n+1))
```

수학 공식 — $O(1)$

```
result = (1 + n) * n / 2
```

5-2. 최댓값 구하기

입력 받기

```
in_value = list(map(int, input().split()))
```

직접 구현 (for 루프)

```
max_val = in_value[0]
for a in in_value[1:]:
    if max_val < a:
        max_val = a
```

5-3. 연결 리스트 구현 (day06)

```
class Node():
    def __init__(self):
        self.data = None
        self.link = None

def printNodes(start):
    current = start
    while current:
        print(current.data, end=' ')
        current = current.link
```

5-4. 스택 구현 (day06)

```
SIZE = 5
stack = [None] * SIZE
top = -1

def push(data):
    global SIZE, stack, top
    if top >= SIZE-1:
        print('Stack Full'); return
    top += 1
    stack[top] = data

def pop():
    global SIZE, stack, top
    if top == -1:
        print('Stack Empty'); return None
    data = stack[top]
    stack[top] = None
    top -= 1
```

```
return data
```

5-5. 괄호 균형 검사 (day07 Quiz)

스택을 이용하여 ()[]의 짝이 맞는지 판단하는 알고리즘

```
bracket_stack = []
answer = True
for j in bracket:
    if j in '([':
        bracket_stack.append(j)
    elif j == ')':
        if not bracket_stack or bracket_stack[-1] != '(':
            answer = False; break
        bracket_stack.pop()
    elif j == ']':
        if not bracket_stack or bracket_stack[-1] != '[':
            answer = False; break
        bracket_stack.pop()
if answer and not bracket_stack:
    print('yes')
else:
    print('no')
```